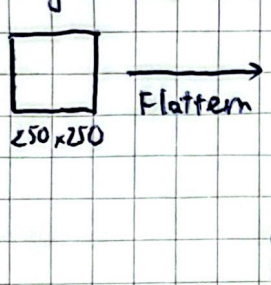
Section 14: Convolutional Neural Network.

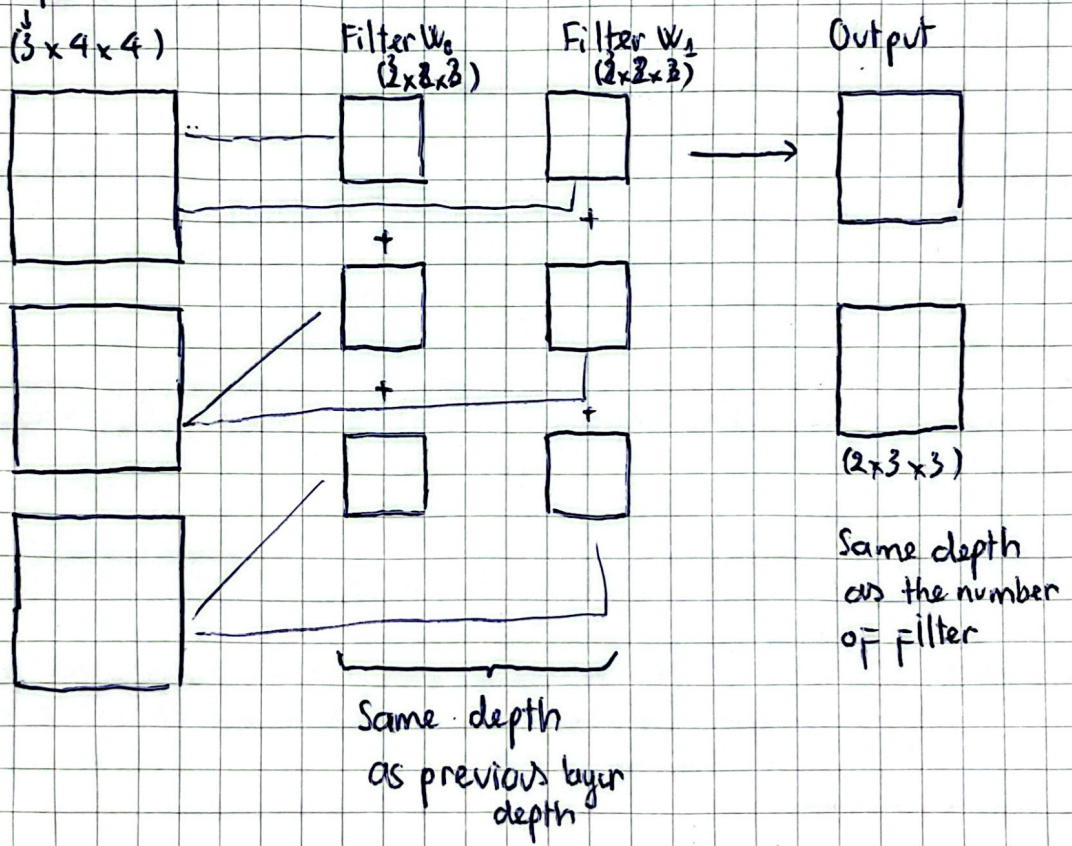
- Why not just flatten the image and use Feed Forward Network?

Imagine we have an image 250 x 250 (= 625000 pixels)

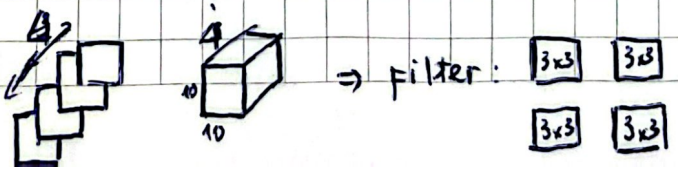


Since we are working with fully connected layers, meaning each neurons is connected to all other neurons
In the hidden layer, each neuron receive 62500 weights. Too many parameters to learn, the network can't handle that and will be overfit!

- We need to think of a way to reduce this : Filter
Old Technique: Edge Detection Filter, Gaussian Blurring Filters
In CNN, we don't set these filters, the network learn these filters using backpropagation.



The depth of each filter is the same as the depth of the previous filter map



KI.ONG

- Pooling and Fully connected

  + Pooling: used to reduce the size of the feature map where still maintaining the importanece features
    Ex: Max-pooling, Ave-pooling

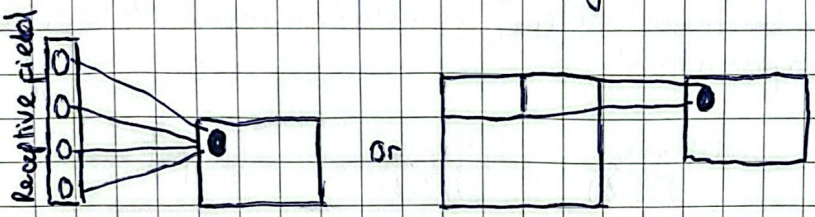
- Output size of the feature map:

$$\text{size} = \frac{\text{input size} - \text{filter size} + 2\text{padding}}{\text{stride}} + 1.$$

  Same padding: You want your feature map size equal to your size
  Vaid padding: _____ less than _____

  for same padding: $\text{padding} = \frac{\text{filter size} - 1}{2}$

- Output size after pooling: $\frac{\text{iput size}}{\text{pooling size}}$



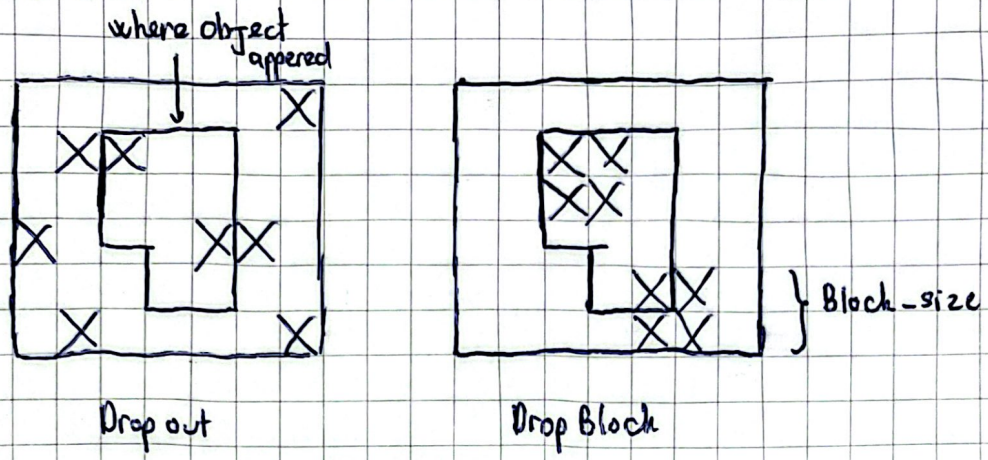
- Shift variance: CNNs are invariant to shifting.

Filters have learned to extract revelant anywhere in the image, no matter where the object happens

- Covariance shift: → Batch Normalization.

- Drop out: less effective, because features are correlated spatially
  So dropping out random neurons is not effective in removing semantic information.
  → DropBlock.

- Drop Block:



Drop out Drop Block

$\gamma = \frac{\text{drop-prob}}{\text{block-size}^2}$ control the number of features to block

Algorithm:

Input: output activation of a layer (A), block_size, γ, mode

if mode == inference then
    return A
endif

Randomly sample mask M: $M_{i,j} \sim \text{Bernoulli}(\gamma)$

For each zero position $M_{i,j}$ create a spatial square mask w center being $M_{i,j}$, set values of M in the square to be zero (choose the center, for short)

Apply the mask: $A = A \times M$ (M = A.shape, only have 1s and 0s)

Normal lize the feature: $A = A \cdot \frac{\text{count}(M)}{\text{count\_ones}(M)}$

- Temperature Soft max: we may want our distribution more or less confident.